R plotting

Supercomputer Center, Westlake University.

https://hpc.westlake.edu.cn

2019/12/03

Plotting

The plotting and graphics engine in R is encapsulated in a few base and recommend packages:

- **graphics:** contains plotting functions for the "base" graphing systems, including *plot*, *hist*, *boxplot* and many others.
- **lattice:** contains code for producing Trellis graphics, which are independent of the "base" graphics system; includes functions like *xyplot*, *bwplot*, *levelplot*
- **grid:** implements a different graphing system independent of the "base" system; the **lattice** package builds on top of **grid**; we seldom call functions from the **grid** package directly
- **grDevices:** contains all the code implementing the various graphics devices, including X11, PDF, PostScript, PNG, etc.

The Process of Making a Plot

- What graphics system will you use: base or grid/lattice? These generally cannot be mixed.
- Base graphics are usually constructed piecemeal, with each aspect of the plot handled separately through a series of function calls; this is often conceptually simpler and allows plotting to mirror the thought process
- Lattice/grid graphics are usually created in a single function call, so all of the graphics parameters have to specified at once; specifying everything at once allows R to automatically calculate the necessary spacings and font sizes.

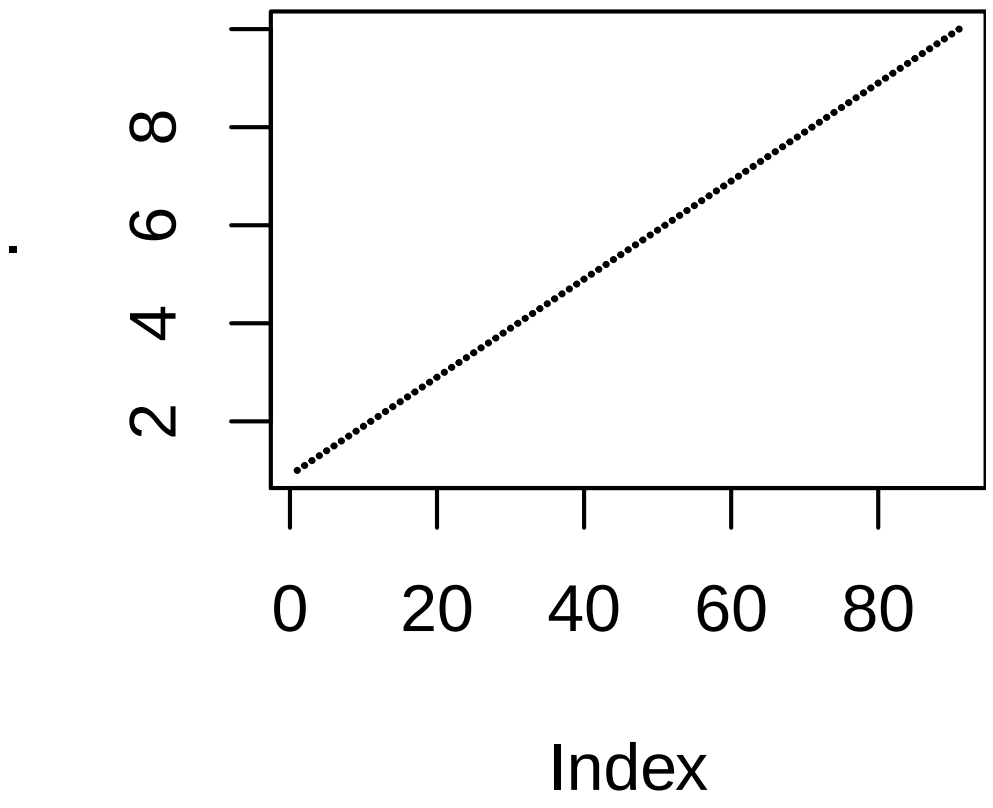
Base Graphics

Base graphics are used most commonly and are a very powerful system for creating 2-D graphics.

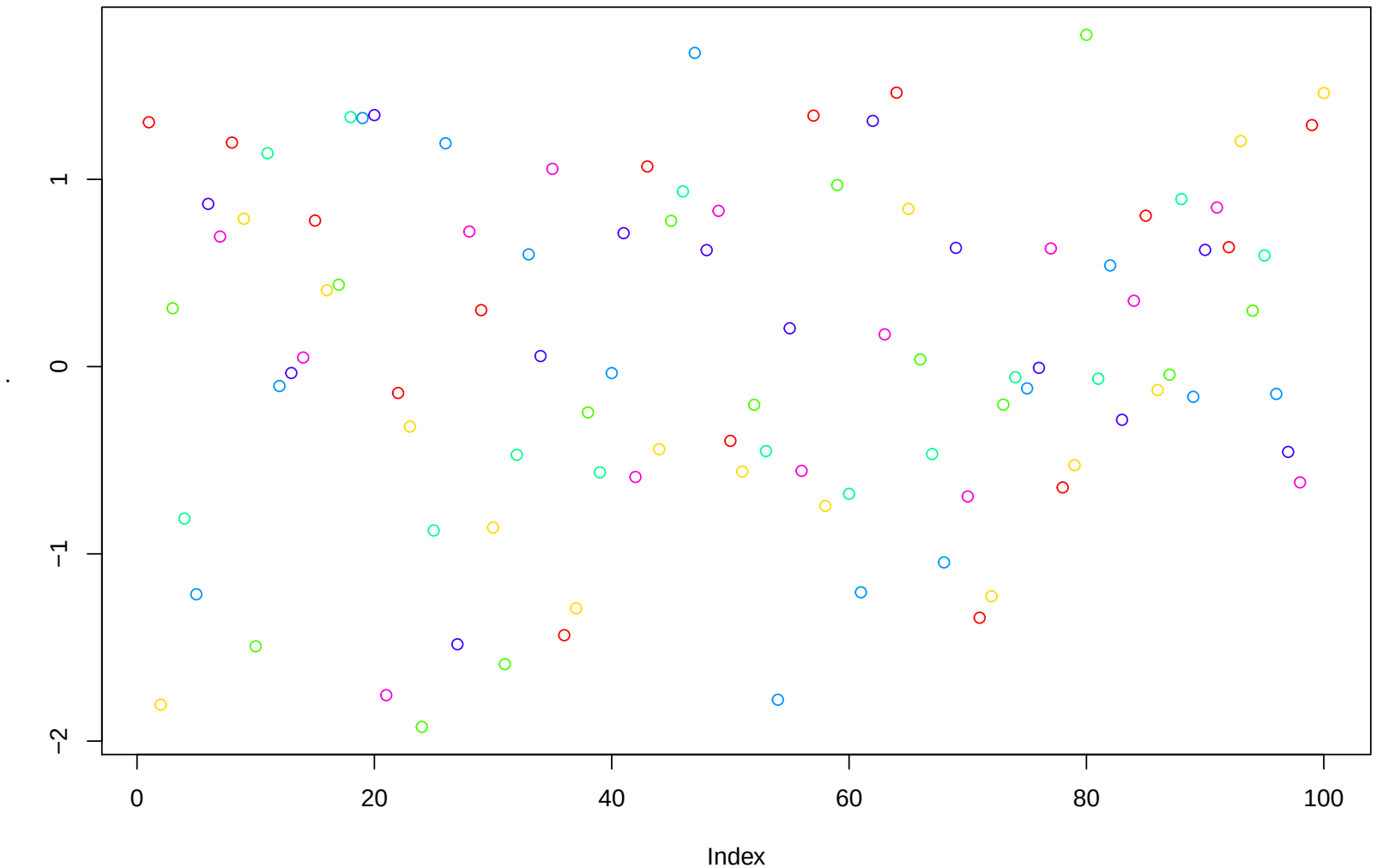
- Calling *plot(x, y)* or *hist(x)* will launch a graphics device (if one is not already open) and draw the plot on the device
- If the arguments to *plot* are not of some special class, then the **default** method for *plot* is called; this function has **many** arguments, letting you set the title, x axis lable, y axis label, etc.
- The base graphics system has **many** parameters that can set and tweaked; these parameters are documented in *?par*; it wouldn't hurt to memorize this help page!

Example for base graphics

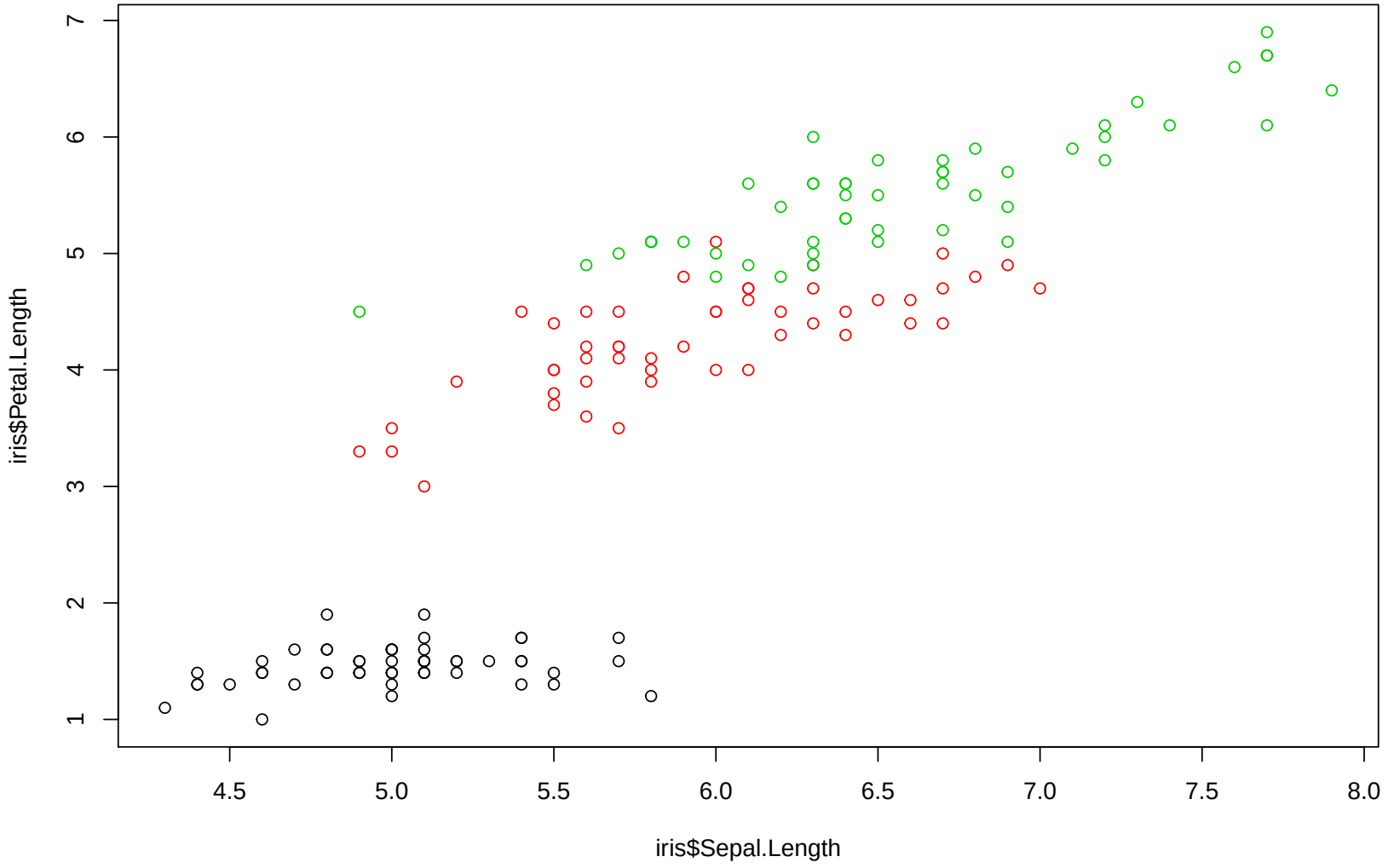
```
> library(pipeR)
> par(mar=c(8,6,1,0))
> seq(1,10,0.1) %>>% plot(cex=0.1)
```



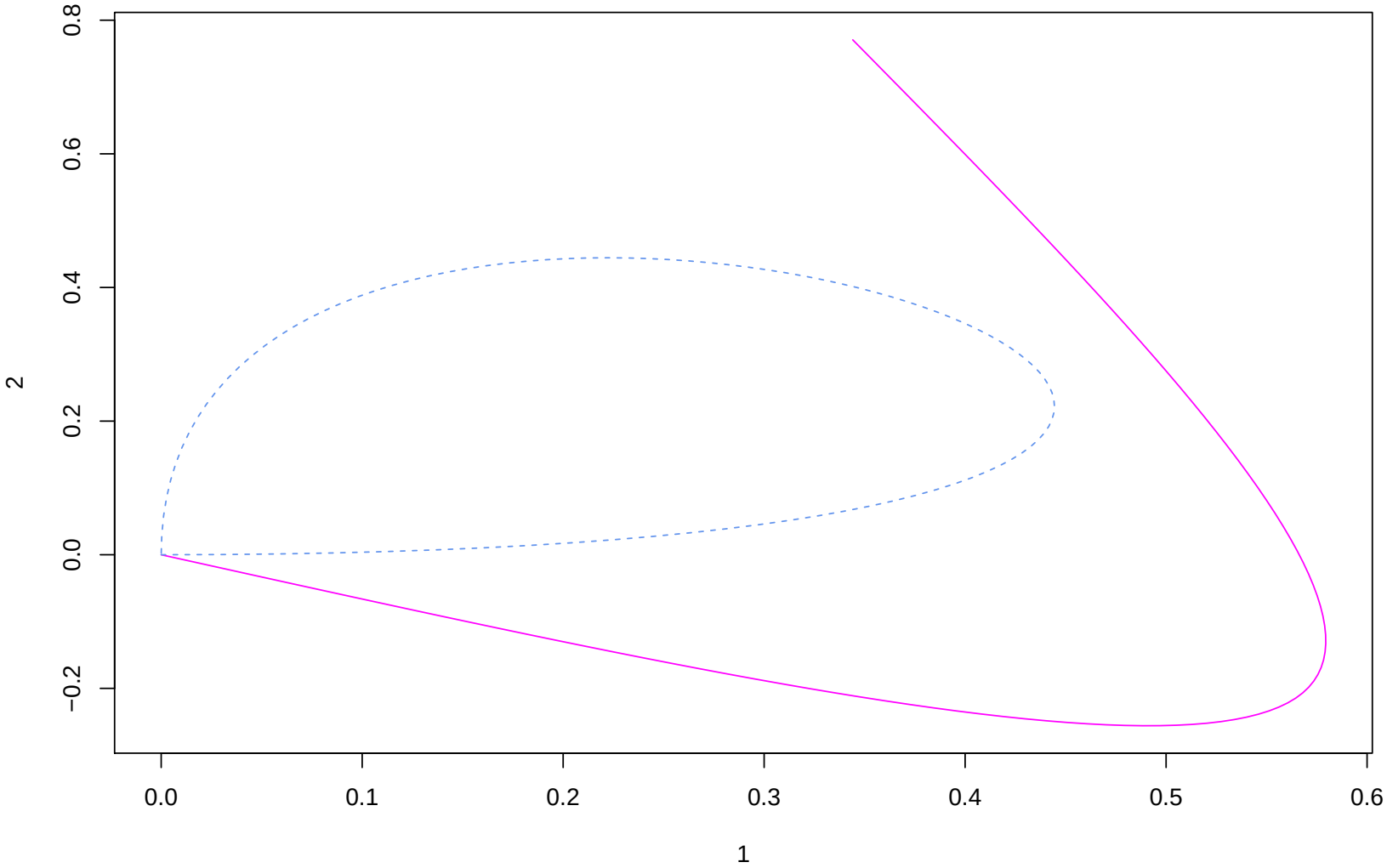
```
> rnorm(100) %>>% plot(col=rainbow(7))
```



```
> plot(iris$Sepal.Length,iris$Petal.Length,col=iris$Species)
```



```
> plot(splines::ns(1:100,2),type="l",col="magenta",lty=1)
> lines(splines::bs(1:100,2),type="l",col="cornflowerblue",lty=2)
```



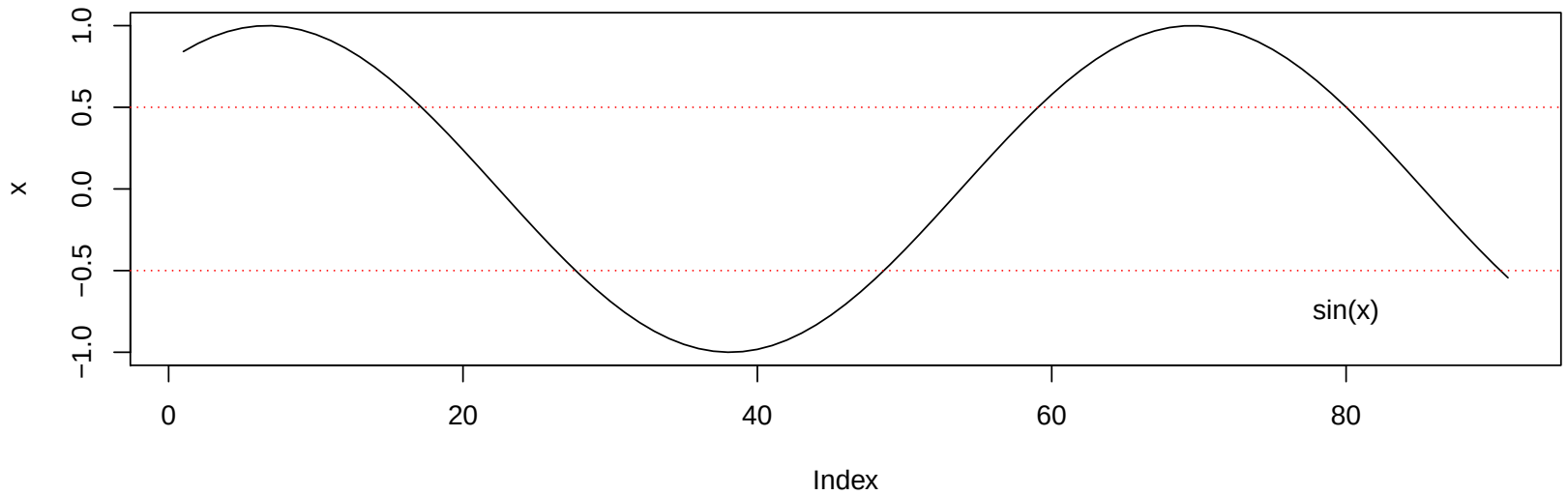
Execise 1

- How to draw a *sin* curve?
- How to add a label "sin(x)" by *text* function?
- How to add cutoff at 0.5 and -0.5 by *abline* function?

Try it!

Code 1

```
> x <- seq(1,10,0.1)
> x <- sin(x)
> plot(x,type="l")
> text(80,-0.75,"sin(x)")
> abline(h=c(-0.5,0.5),col="red",lty=3)
```



Lattice Functions

- *xyplot*: this is the main function for creating scatterplots
- *bwplot*: box-and-whiskers plot ("boxplots")
- *histogram*: histograms
- *stripplot*: like a boxplot but with actual points
- *dotplot*: plot dots on "violin strings"
- *splom*: scatterplot matrix; like *pairs* in base graphics system
- *levelplot*, *contourplot*: for plotting "image" data

Lattice Functions

Lattice functions generally take a formula for their first argument, usually of the form

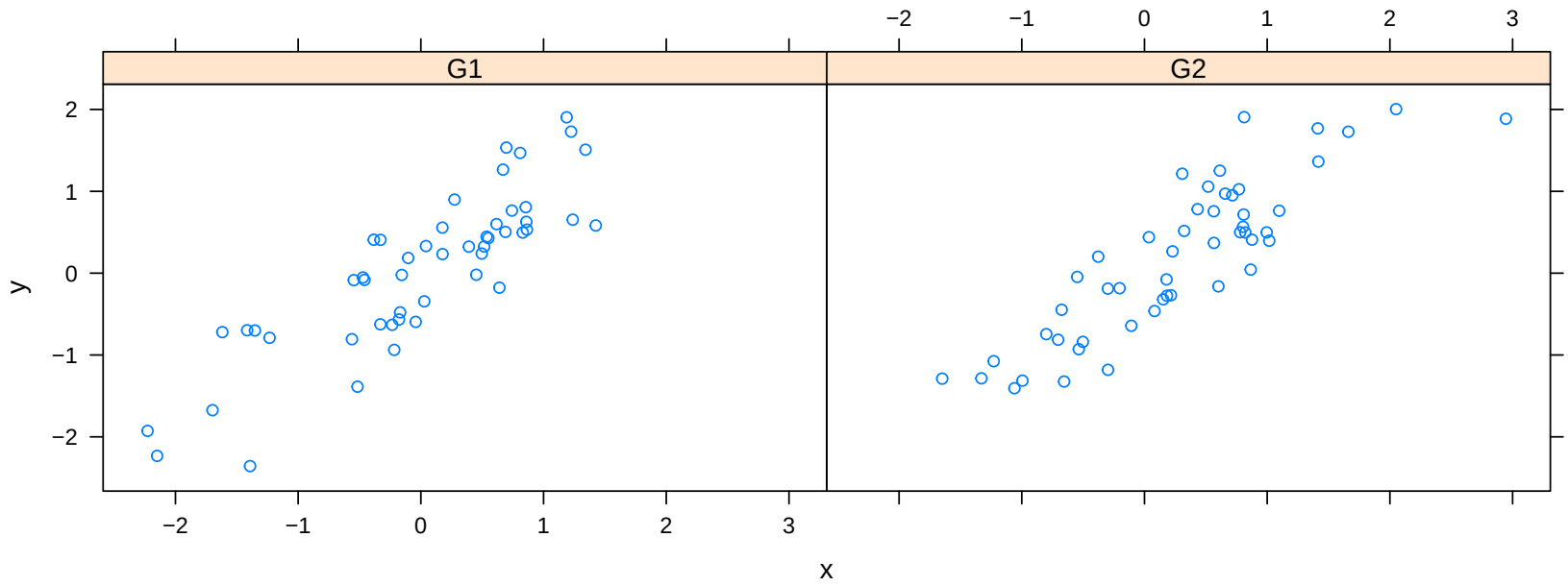
```
y ~ x | f * g
```

- On the left of the ~ is the y variable, on the right is the x variable
- After the | are *conditioning variables* – they are optional; the * indicates an interaction
- The second argument is the data frame or list from which the variables in the formula should be obtained
- If no data frame or list is passed, then the parent frame is used
- If no other arguments are passed, there are defaults that can be used

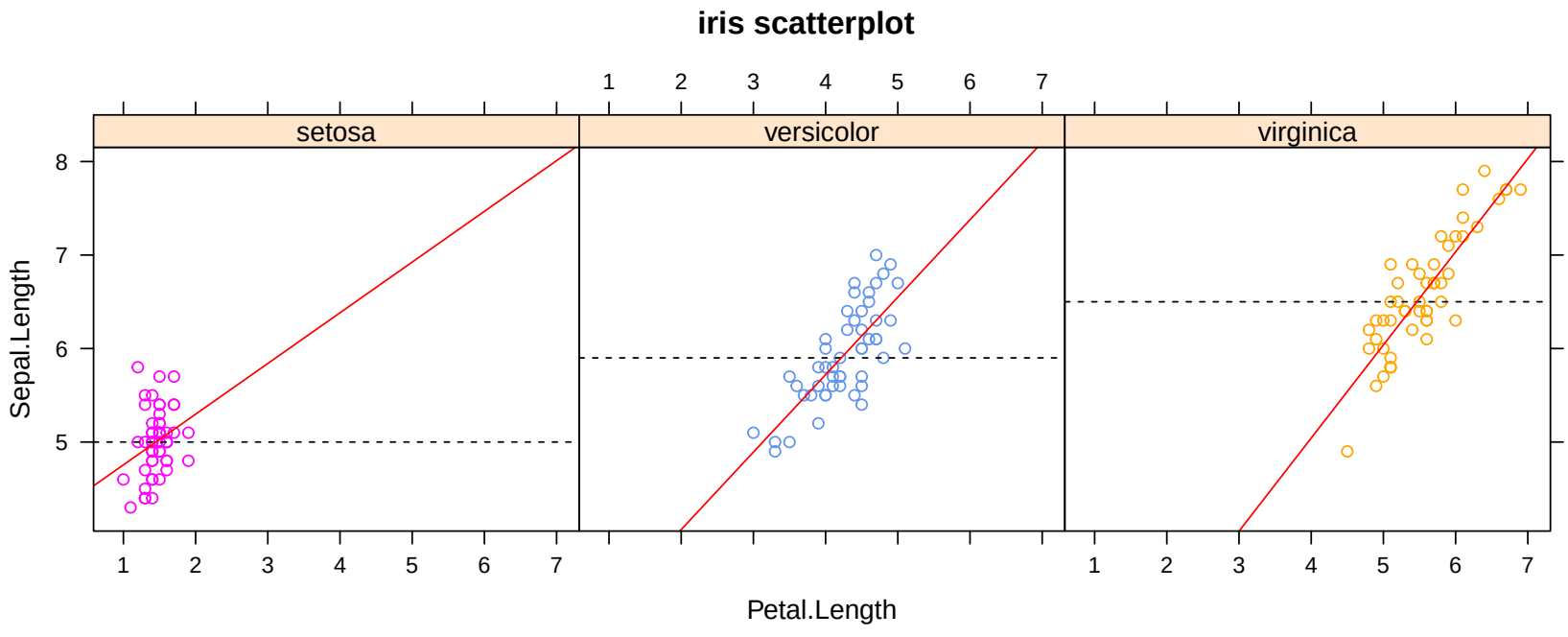
Lattice Panel Functions

Lattice functions have a panel function which controls what happens inside each panel of the entire plot.

```
> library(lattice)
> x <- rnorm(100)
> y <- x + rnorm(100,sd=0.5)
> f <- gl(2,50,labels=c("G1","G2"))
> xyplot(y ~ x | f)
```



```
> xyplot(Sepal.Length ~ Petal.Length | Species,data=iris,
+        col=c("magenta","cornflowerblue","orange"),group=Species,
+        main="iris scatterplot",
+        panel=function(x,y,...) {
+          panel.xyplot(x,y,...)
+          panel.abline(h=median(y),lty=2)
+          panel.lmline(x,y,col="red")
+        })
```



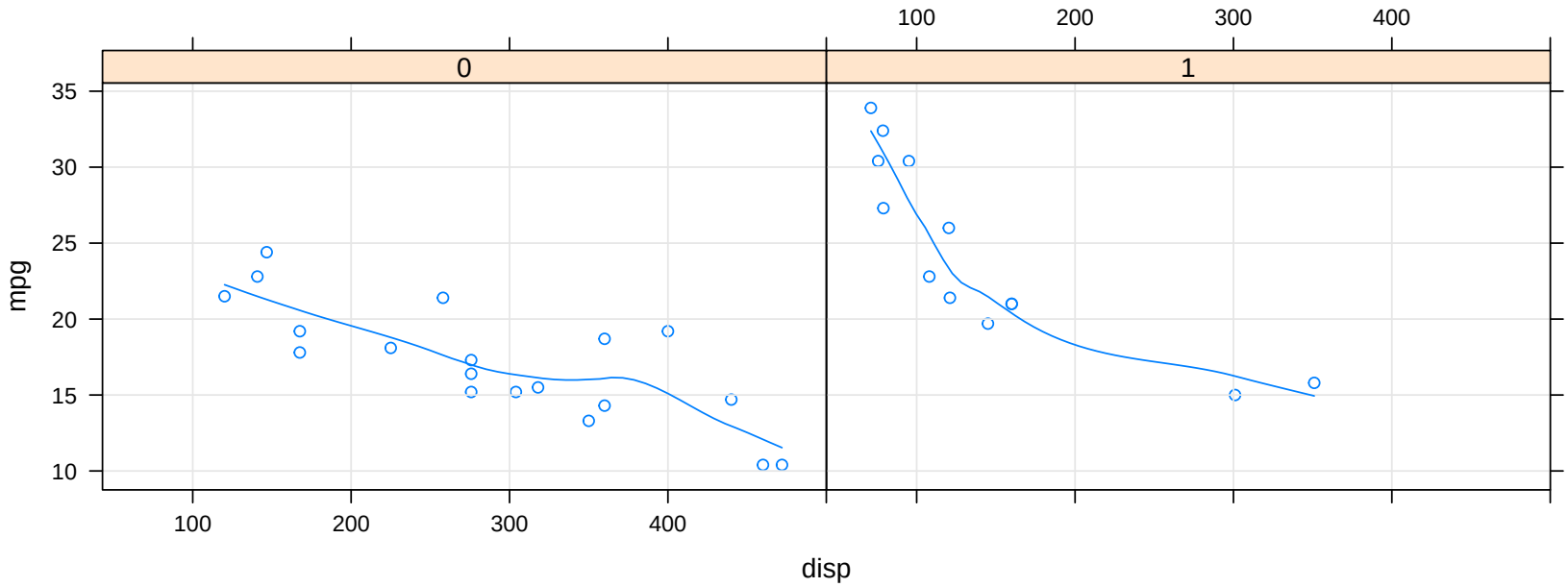
Execise 2

- How to draw a "mpg vs disp by am" plot for mtcars data with points, smooth curve and grid by *xyplot*?

Note: *panel.loess* and *panel.grid* can make a smooth curve and grid.

Code 2

```
> xyplot(mpg ~ disp | factor(am), data=mtcars,
+        panel=function(x,y,...) {
+          panel.xyplot(x,y,...)
+          panel.grid(h=-1,v=-1)
+          panel.loess(x,y,...)
+        })
```



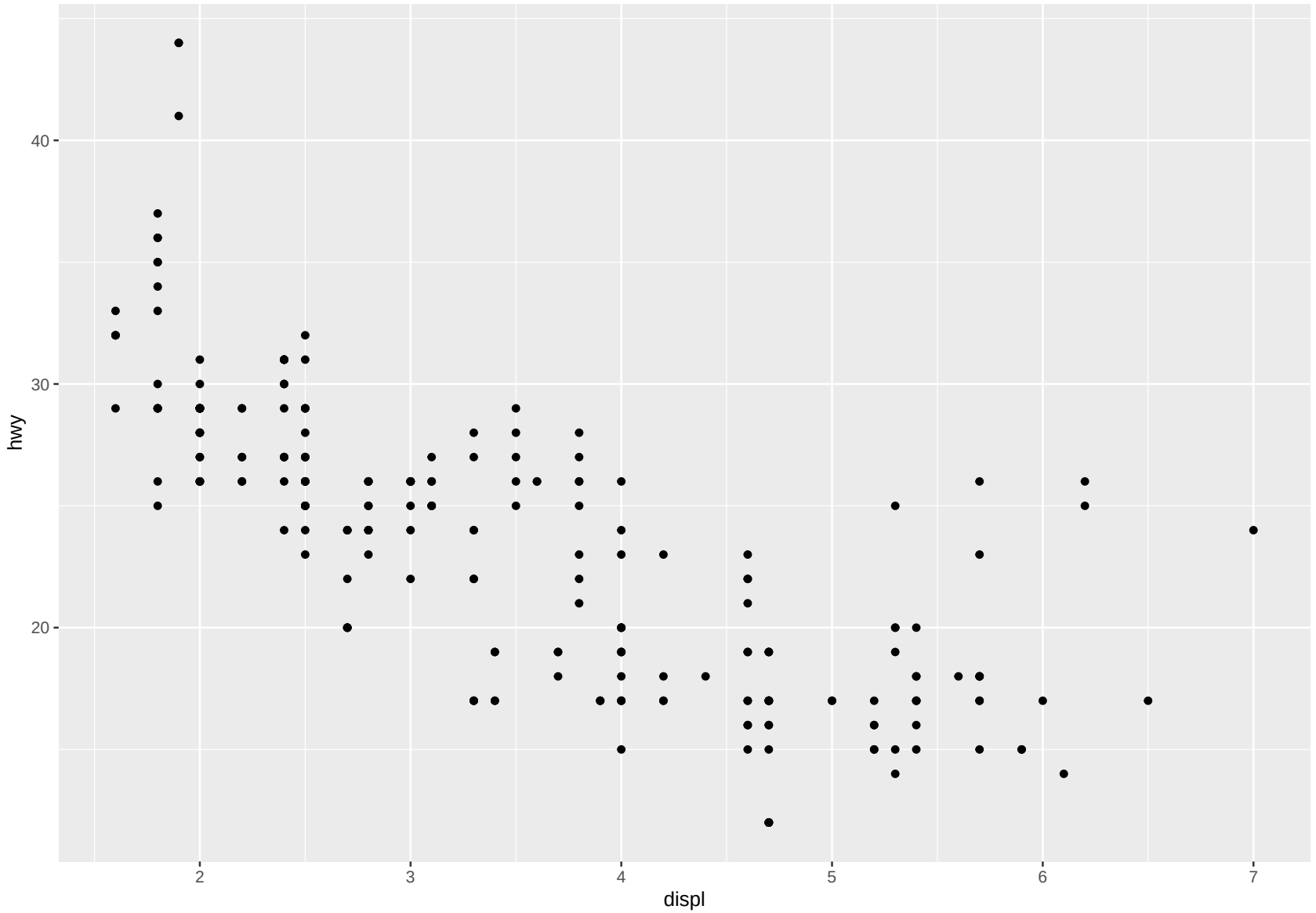
ggplot2

- An implementation of the *Grammar of Graphics* by Leland Wilkinson
- Written by Hadley Wickham (while he was a graduate student at Iowa State)
- A "third" graphics system for R (along with **base** and **lattice**)
- Available from CRAN via *install.packages*()
- Web site: https://ggplot2.tidyverse.org

The Basics: qplot()

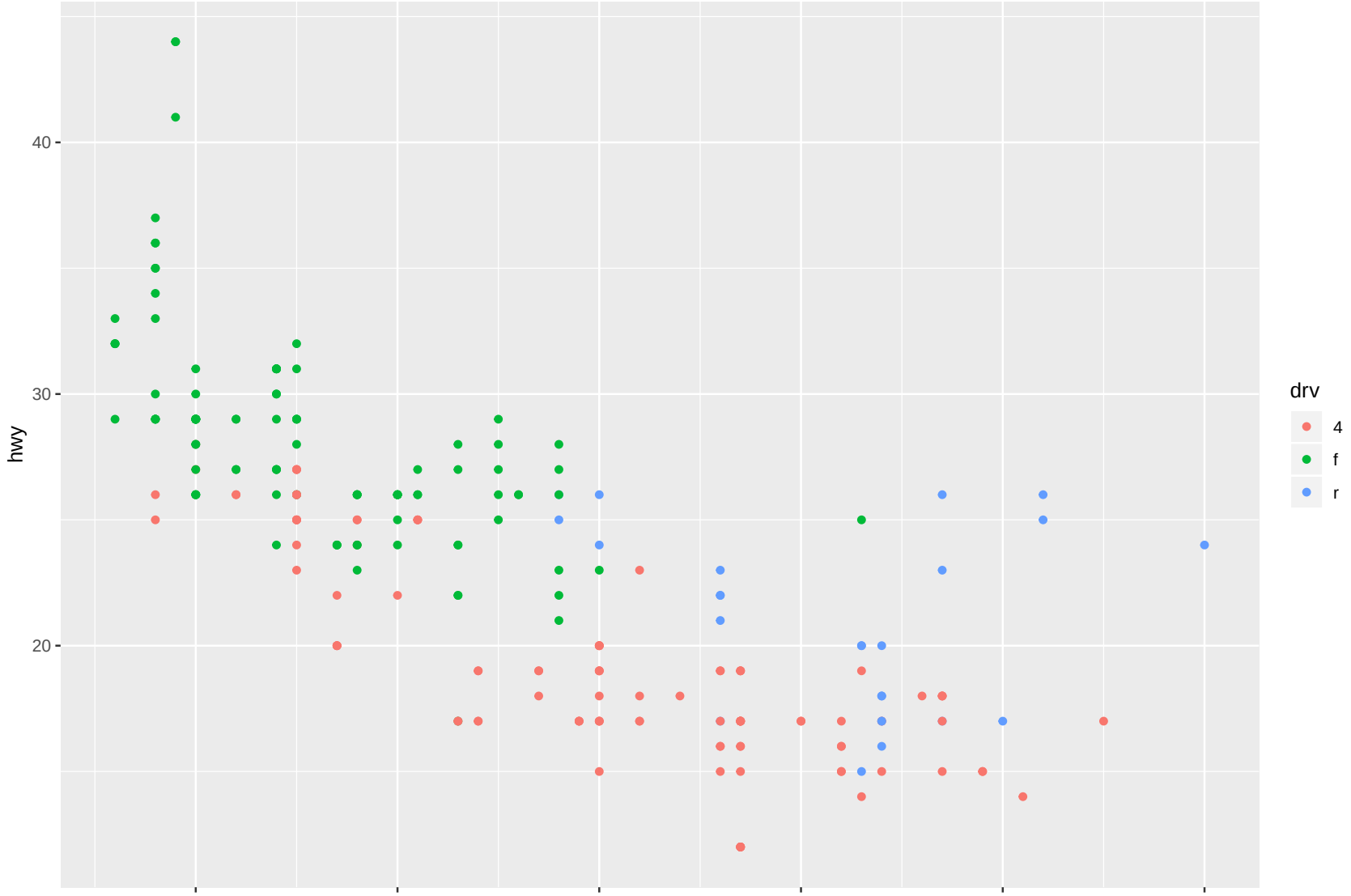
- Works much like the *plot* function in base graphics system
- Looks for data in a data frame, similar to lattice
- Plots are made up of *aesthetics* (size, shape, color) and *geoms* (points, lines)

```
> library(ggplot2)
> qplot(displ,hwy,data=mpg)
```



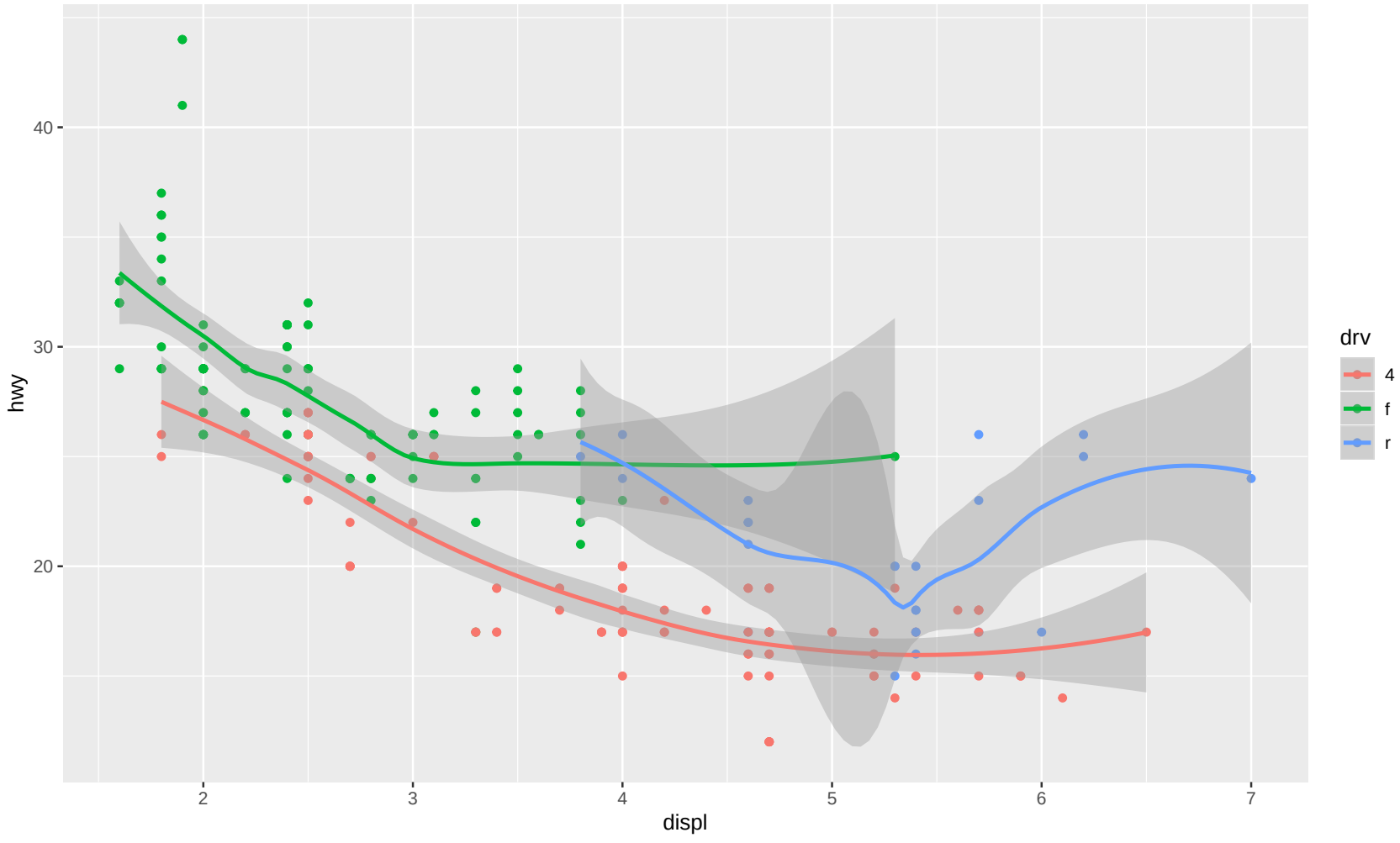
Modifying aesthetics

```
> qplot(displ,hwy,data=mpg,color=drv)
```



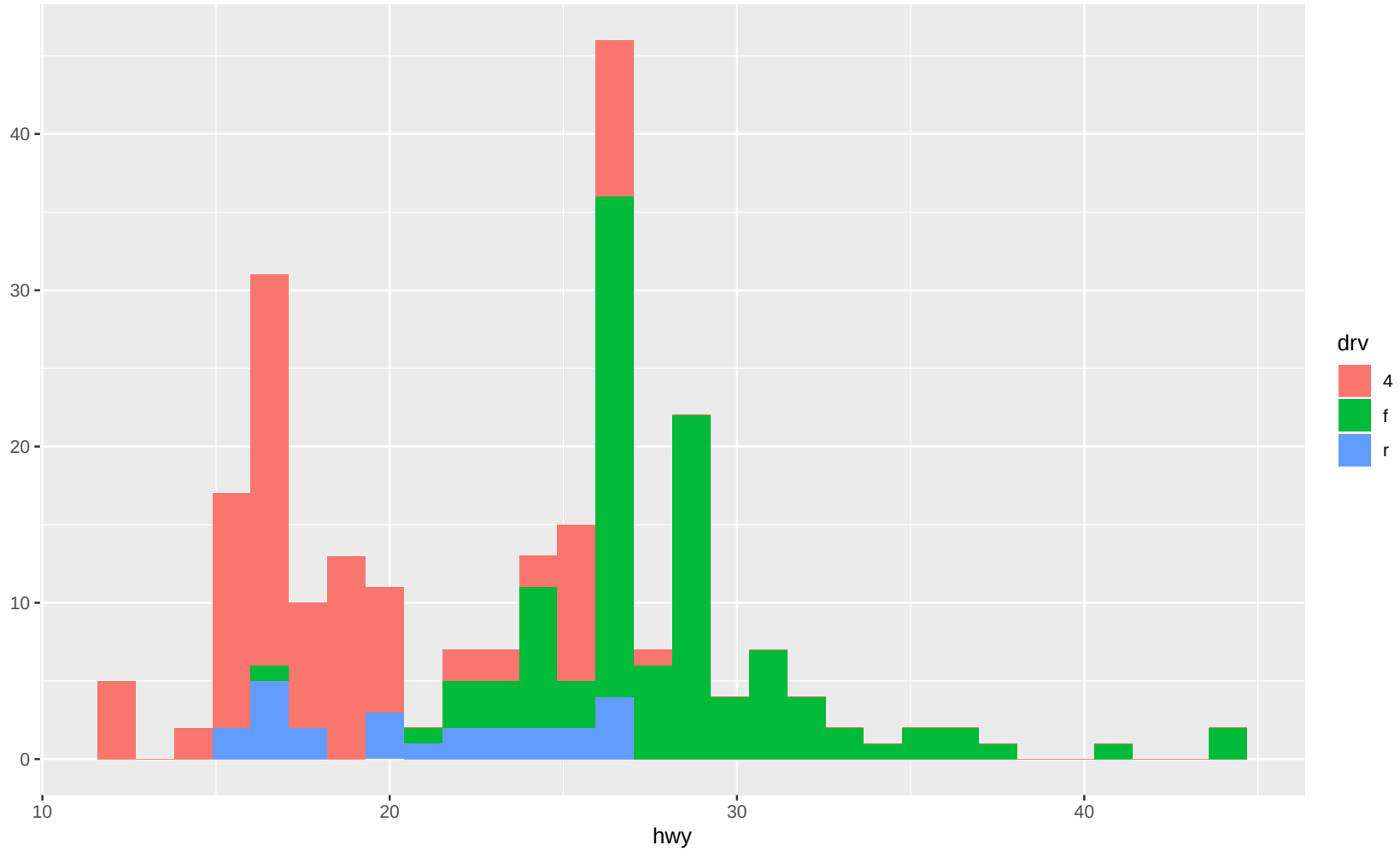
Adding a geom

```
> qplot(displ,hwy,data=mpg,color=drv,geom=c("point","smooth"))
```



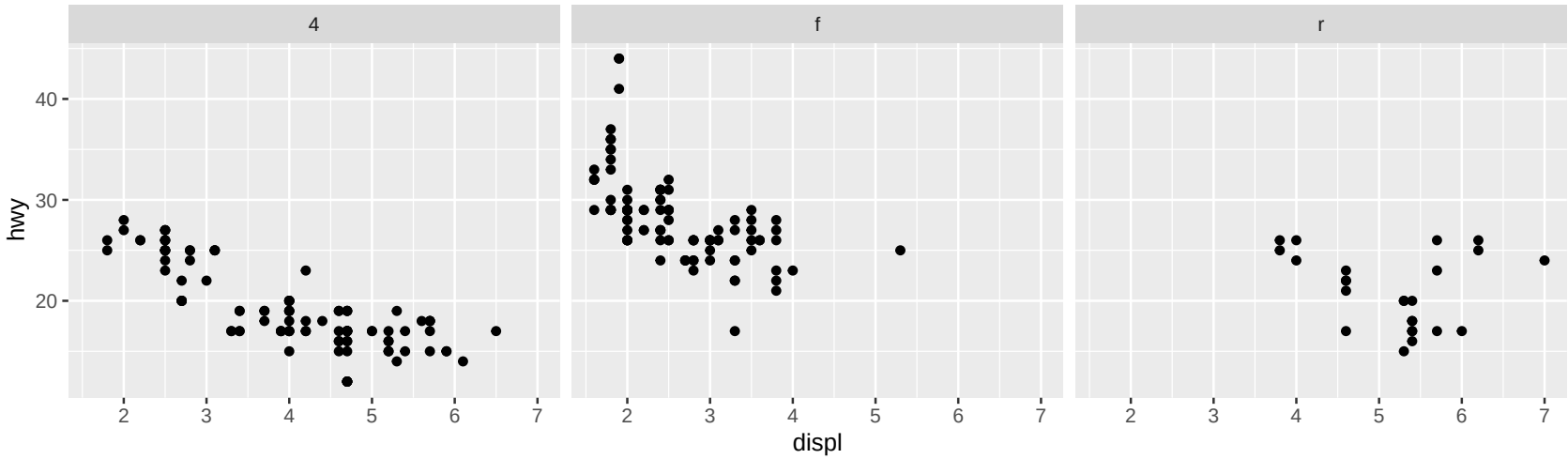
Histograms

```
> qplot(hwy,data=mpg,fill=drv)
```

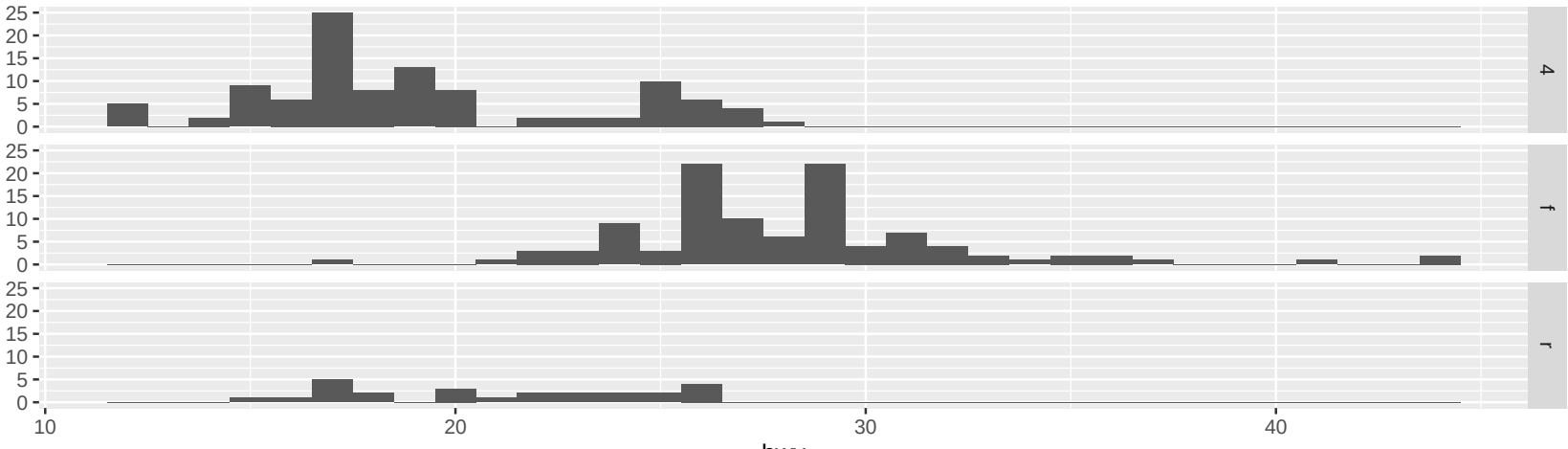


Facets

```
> qplot(displ,hwy,data=mpg,facets=.~drv)
```

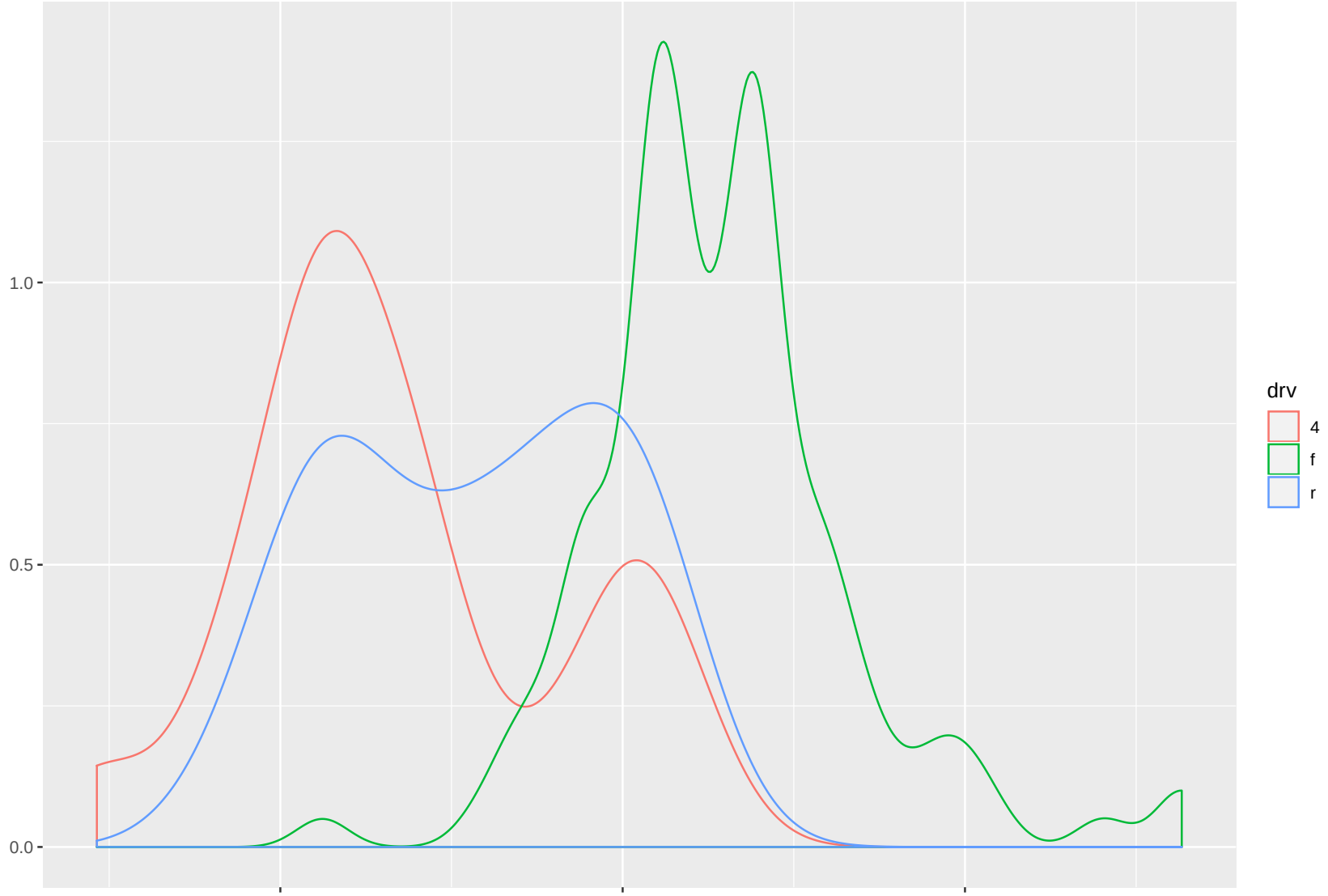



```
> qplot(hwy,data=mpg,facets=drv~.,binwidth=1)
```



Density

```
> qplot(sqrt(hwy),data=mpg,geom="density",color=drv)
```



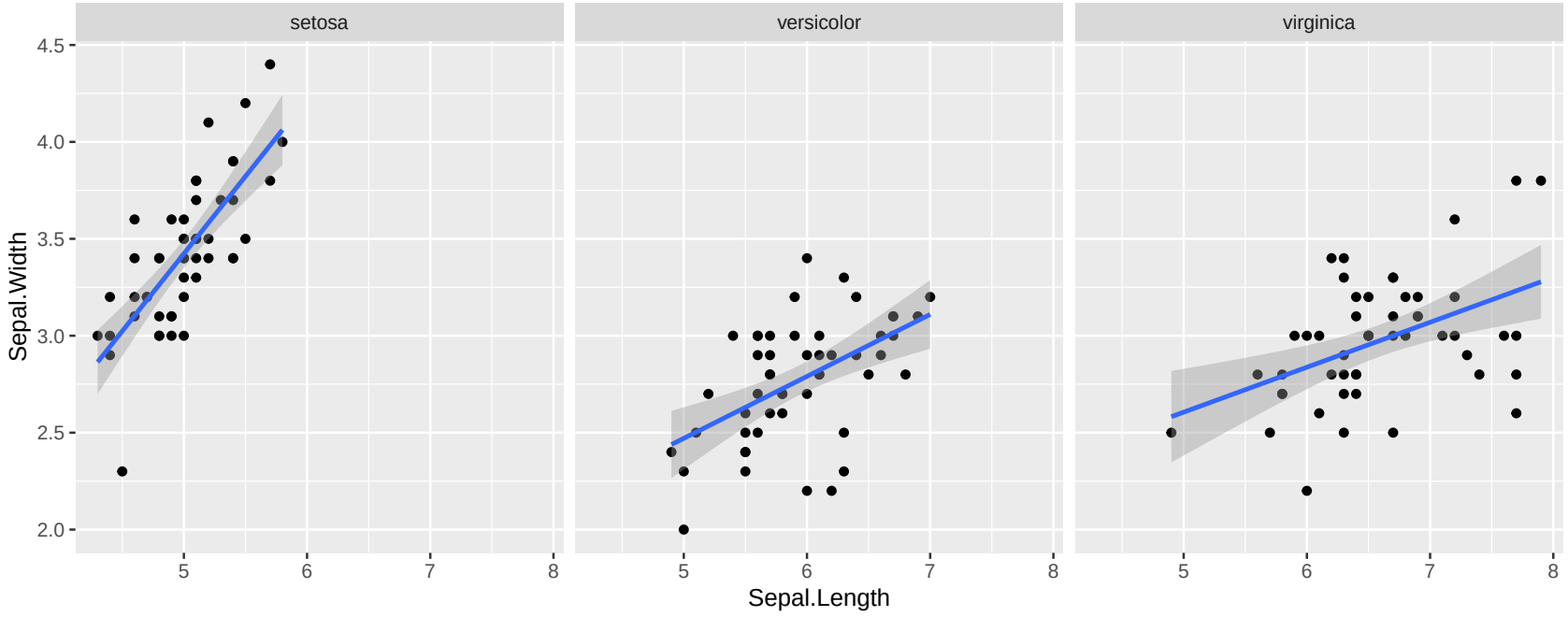
Power weapon: ggplot()

Basic components of a ggplot2 Plot:

- A **data frame**
- **aesthetic mappings:** how data are mapped to color, size
- **geoms:** geometric objects like points, lines, shapes
- **facets:** for conditional plots
- **stats:** statistical transformations like binning, quantiles, smoothing
- **scales:** what scale an aesthetic map uses (example: male = red, female = blue)
- **coordinate system**

- Plots are built up in **layers**
- Plot the data
- Overlay a summary
- Metadata and annotation

```
> qplot(Sepal.Length, Sepal.Width,data=iris,facets=.~Species,
+       geom=c("point","smooth"),method="lm")
```



Building up in **layers**

```
> head(iris)
```

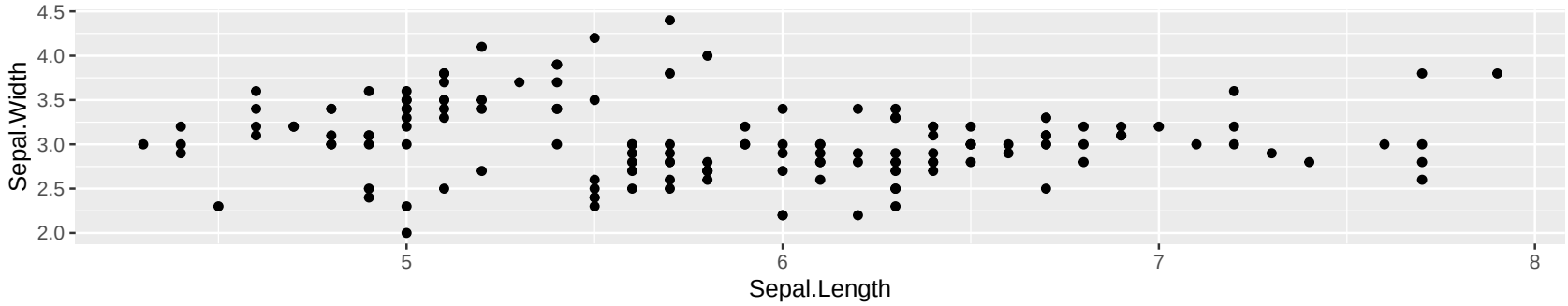
```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1          5.1         3.5          1.4         0.2  setosa
## 2          4.9         3.0          1.4         0.2  setosa
## 3          4.7         3.2          1.3         0.2  setosa
## 4          4.6         3.1          1.5         0.2  setosa
## 5          5.0         3.6          1.4         0.2  setosa
## 6          5.4         3.9          1.7         0.4  setosa
```

```
> g <- ggplot(iris,aes(Sepal.Length,Sepal.Width))
```

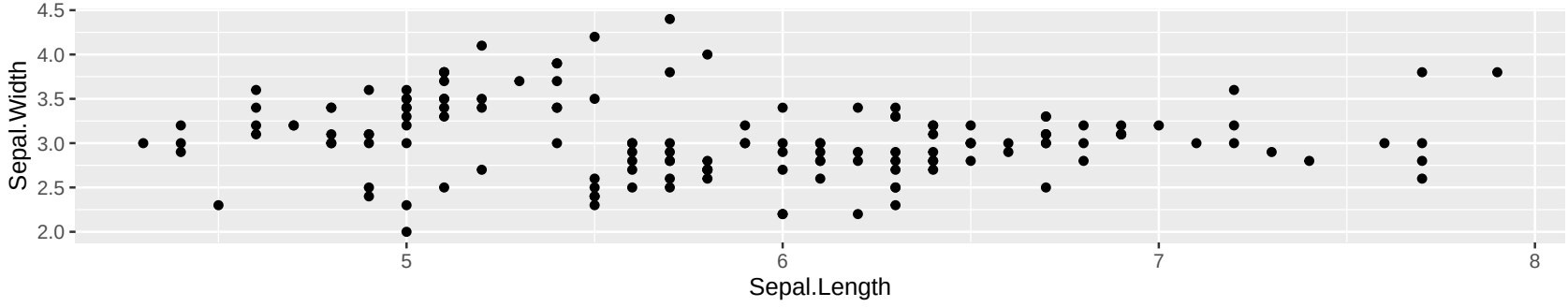
No Plot Yet!

Point layer

```
> g + geom_point()
```

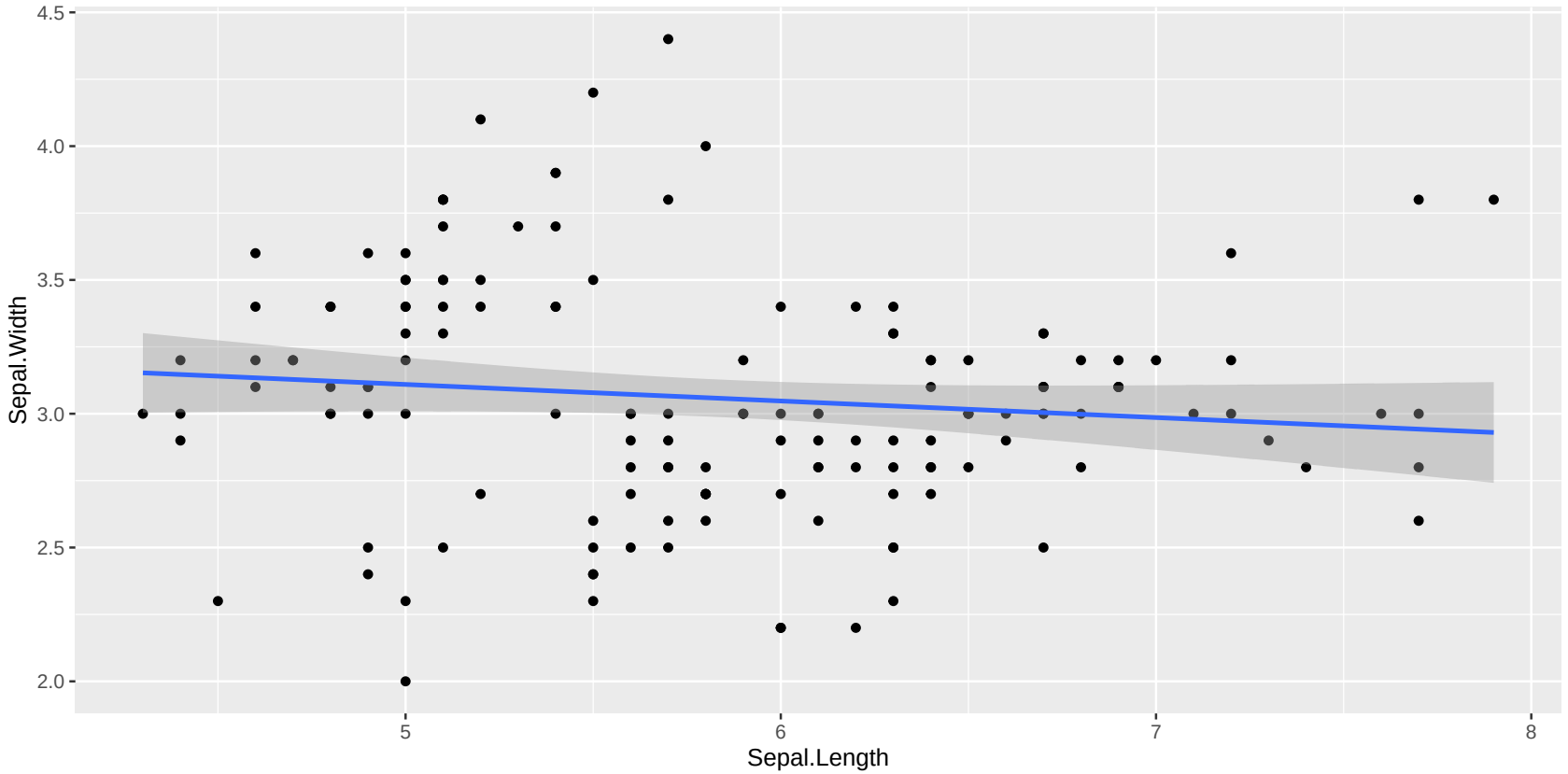



```
> p <- g + geom_point()
> print(p)
```



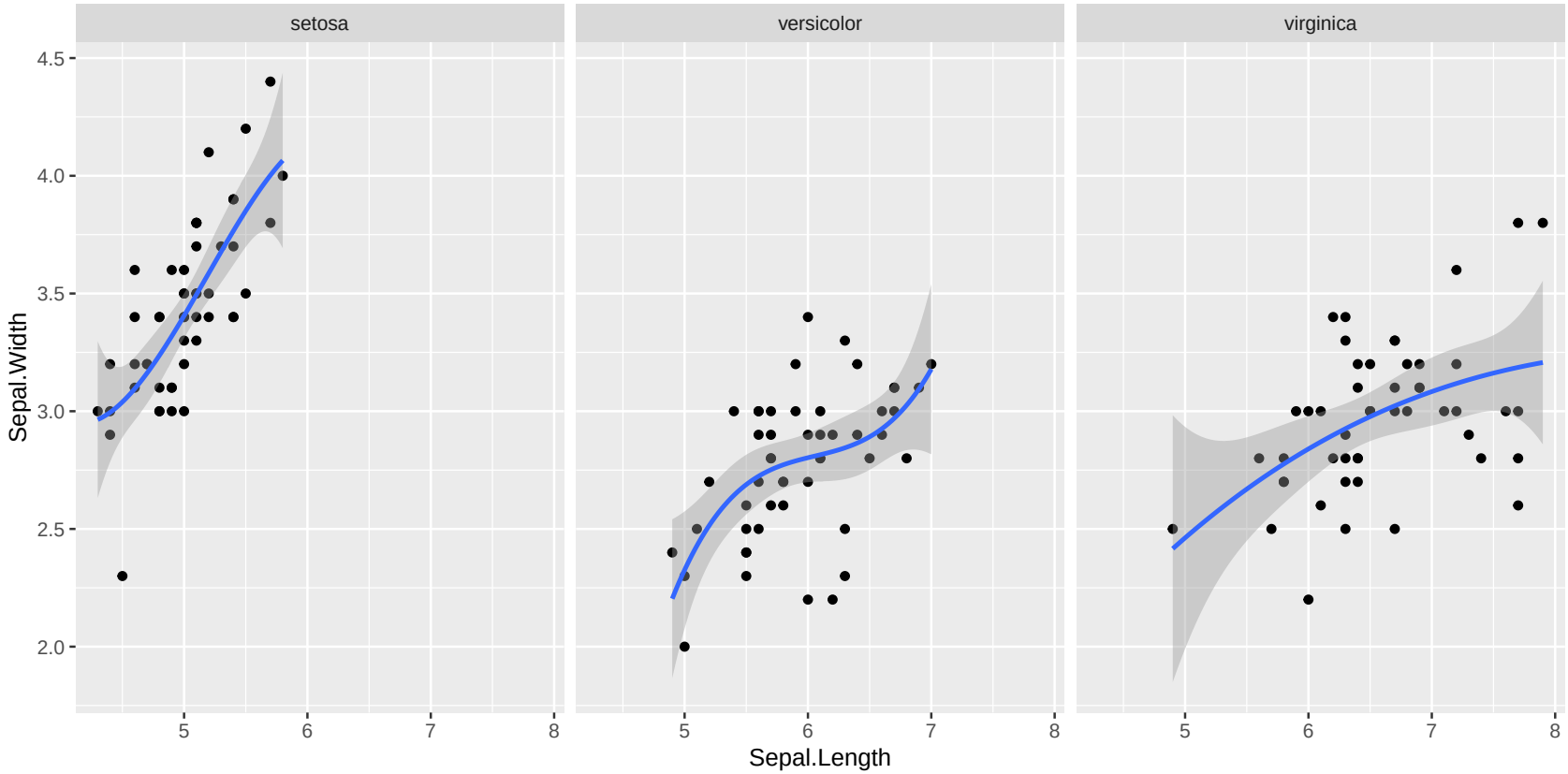
Adding more layers: smooth

```
> g + geom_point() + geom_smooth(method="lm")
```



Adding more layers: facets

```
> g + geom_point() +
+   geom_smooth(method="lm",formula=y~splines::bs(x,3)) +
+   facet_grid(.~Species)
```

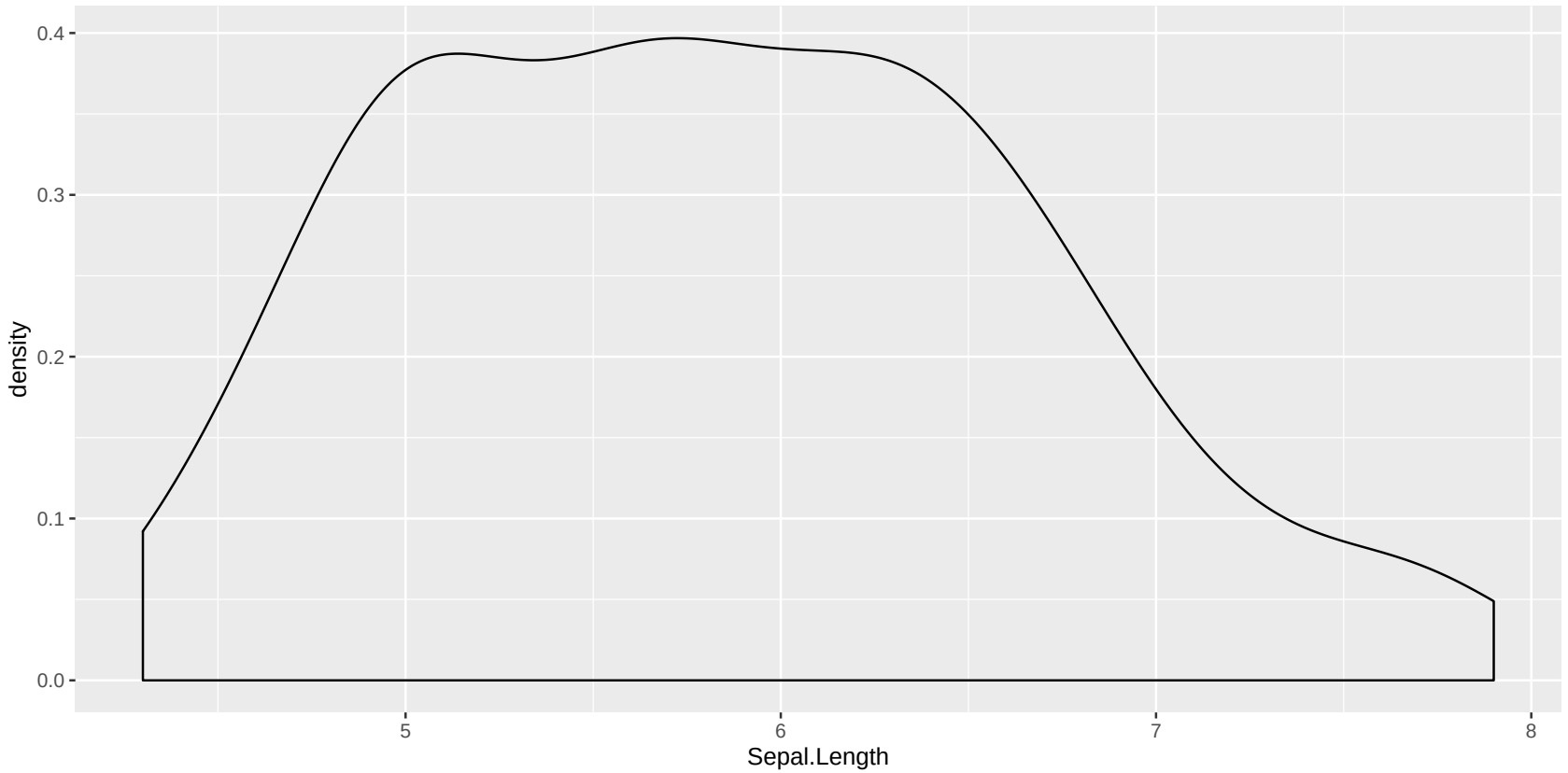


Execise 3

- How to add a density layer of Sepal.Length from iris data set by geom_density?

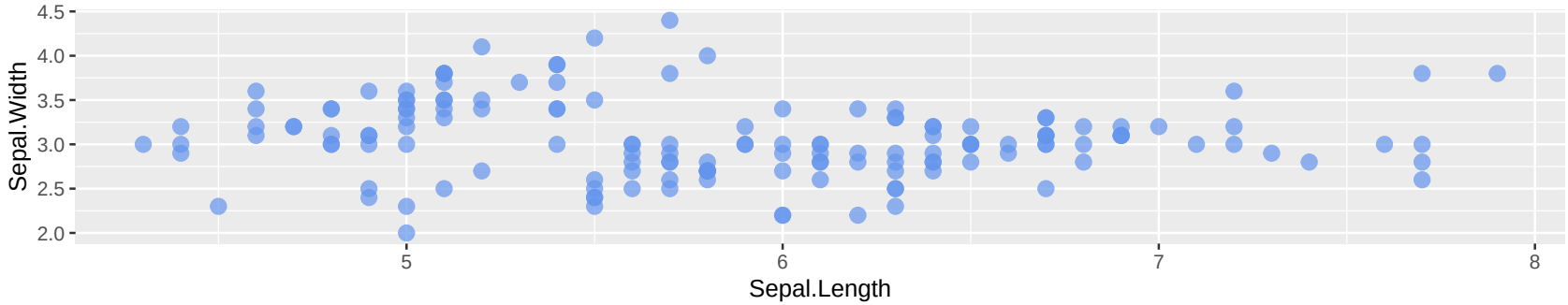
Code 3

```
> ggplot(iris,aes(Sepal.Length)) + geom_density()
```

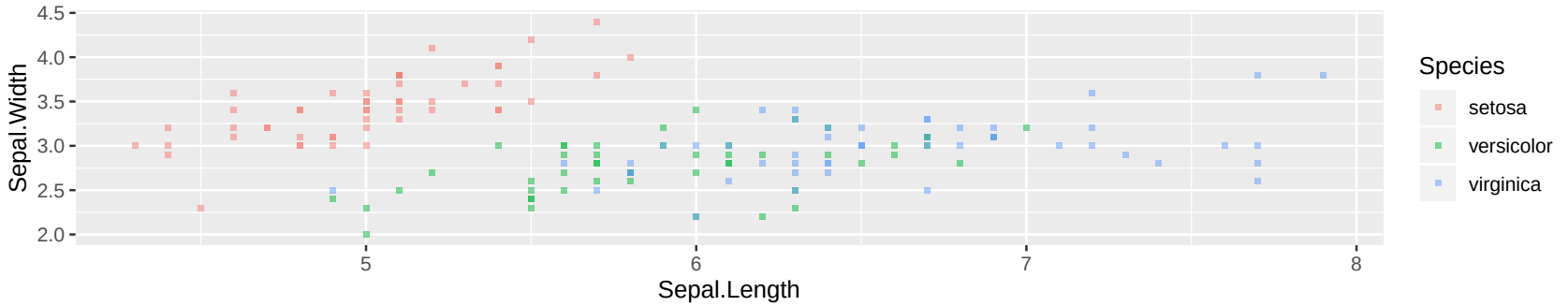


Modifying aesthetics

```
> g + geom_point(color="cornflowerblue",size=3,alpha=0.7,shape=19)
```

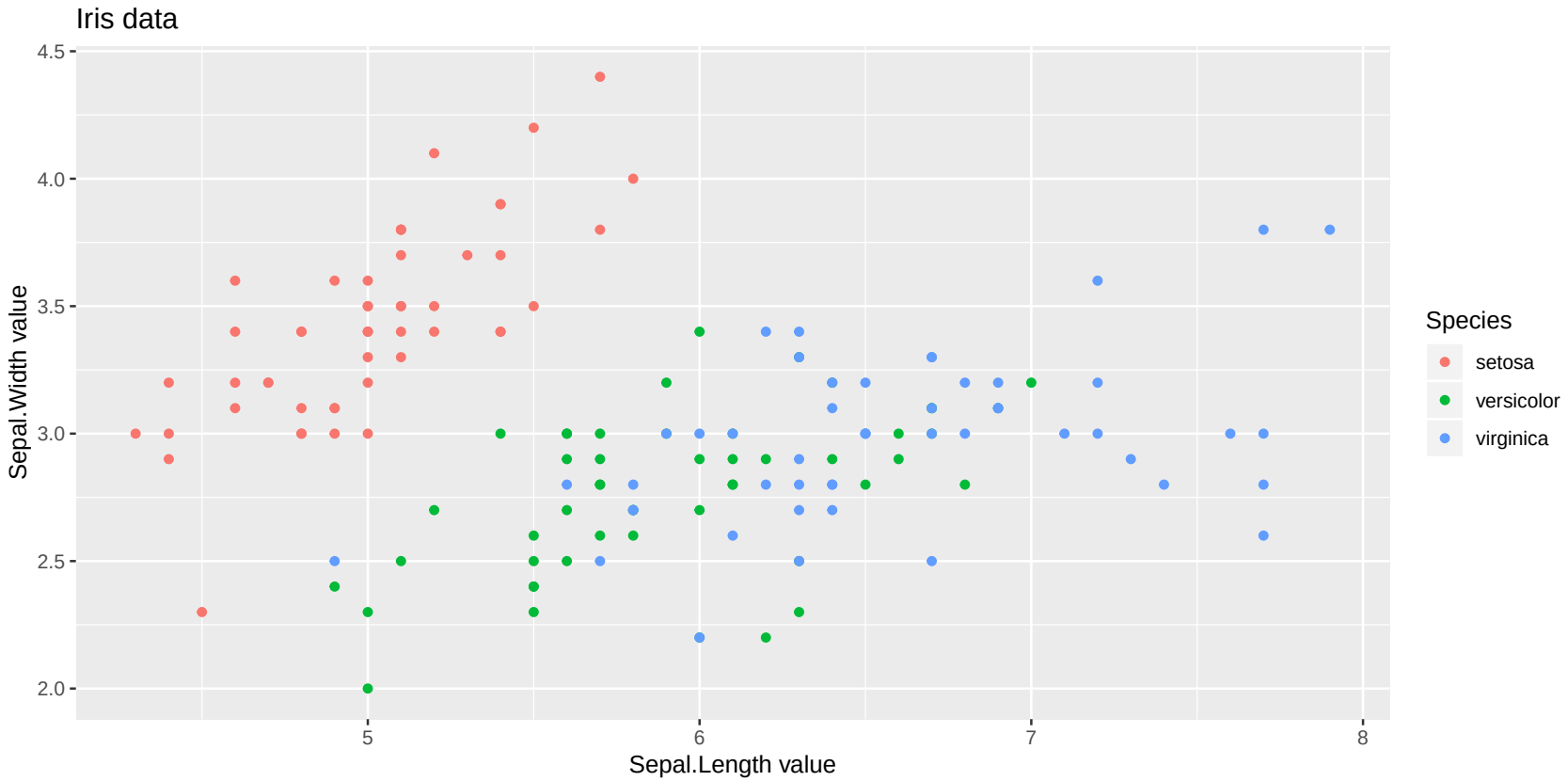



```
> g + geom_point(aes(color=Species),size=1,alpha=0.5,shape=15)
```



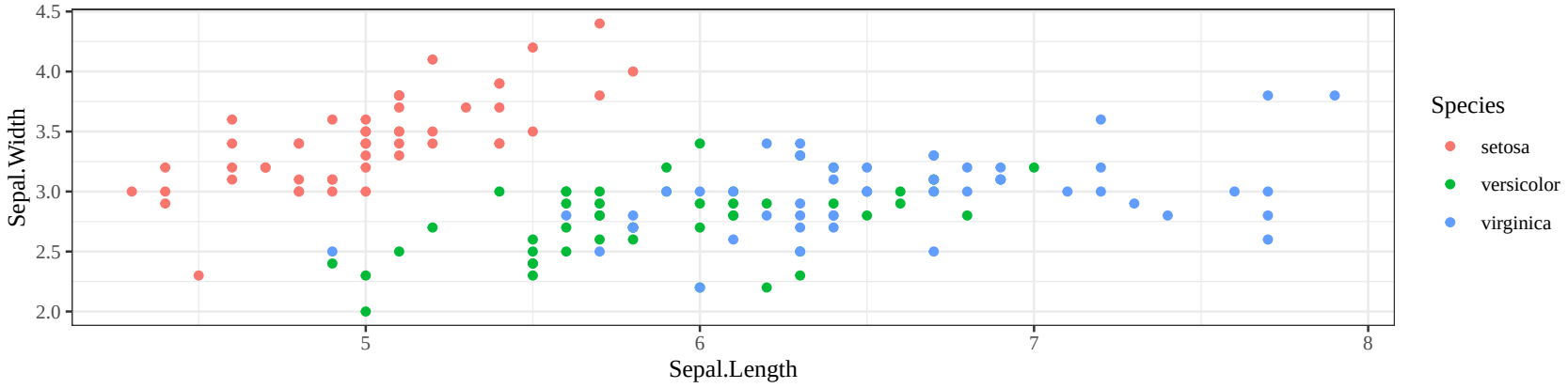
Modifying labels

```
> g + geom_point(aes(color=Species)) + labs(title="Iris data") +
+   labs(x="Sepal.Length value",y="Sepal.Width value")
```

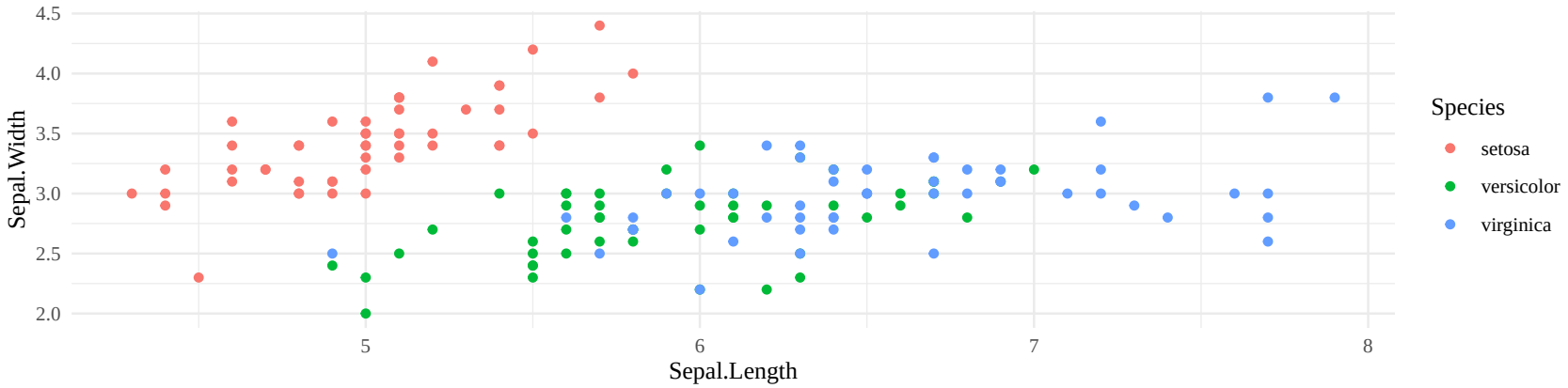


Changing the theme

```
> g + geom_point(aes(color=Species)) + theme_bw(base_family="Times")
```

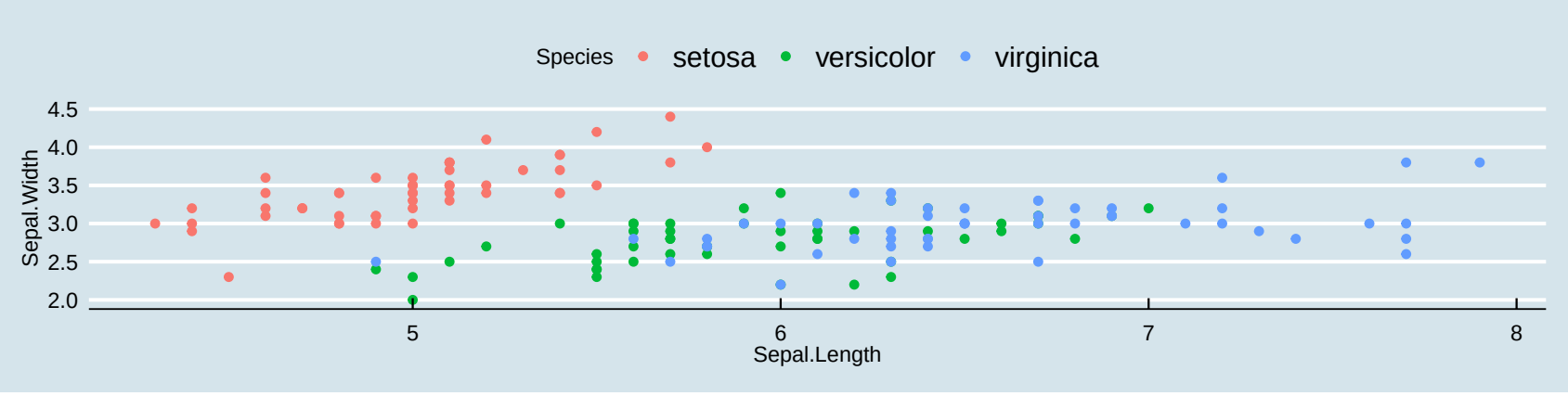



```
> g + geom_point(aes(color=Species)) +
+   theme_minimal(base_family="Times")
```

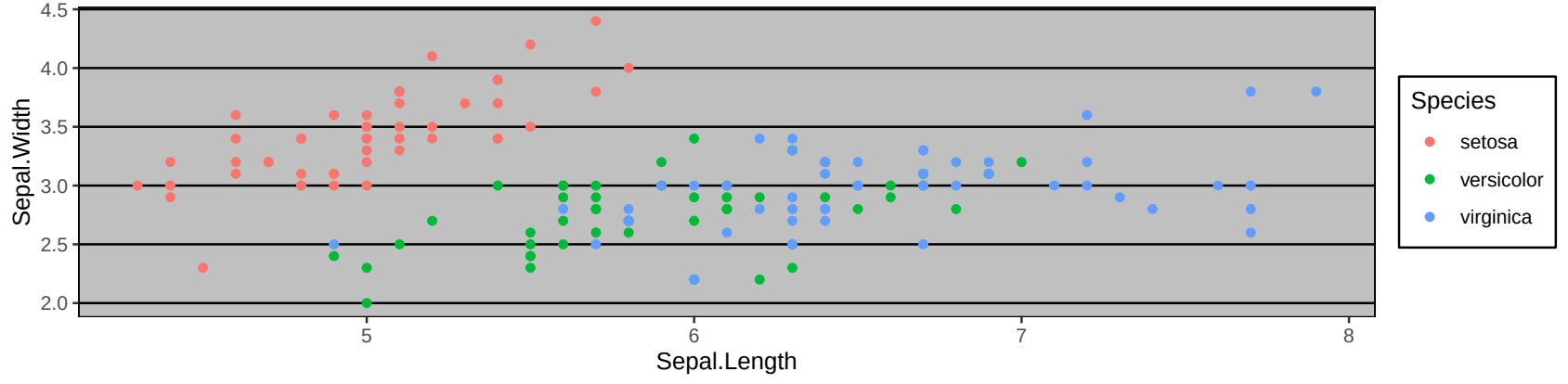


ggthemes package gives more themes to ggplot2

```
> library(ggthemes)
> g + geom_point(aes(color=Species)) + theme_economist()
```

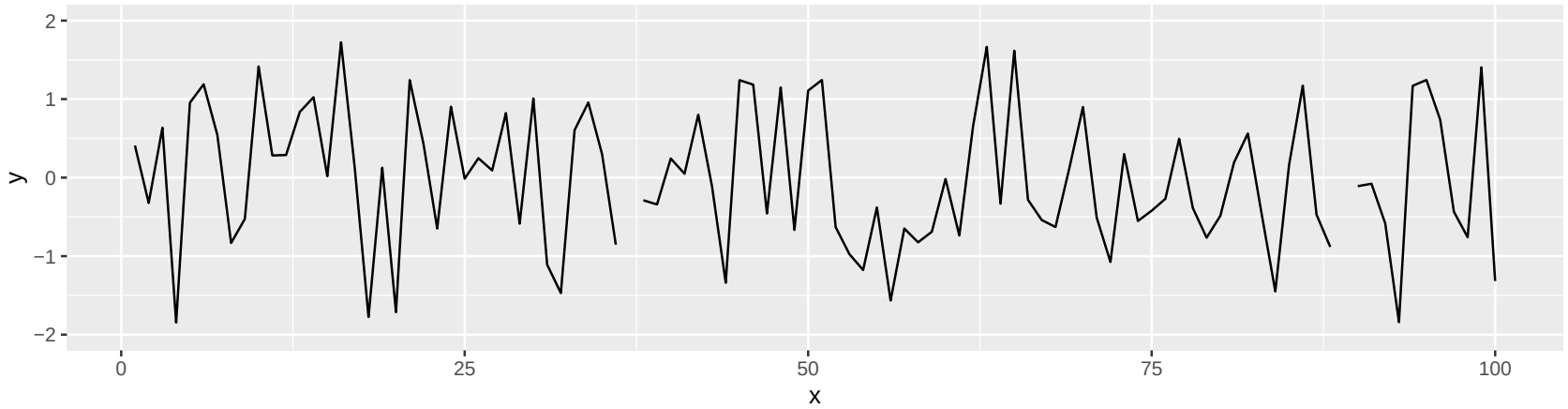



```
> g + geom_point(aes(color=Species)) + theme_excel()
```

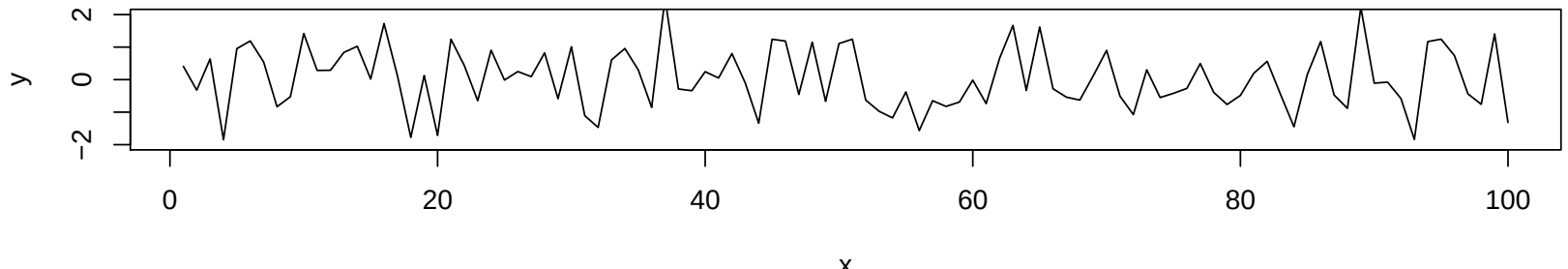


Be careful: axis limits

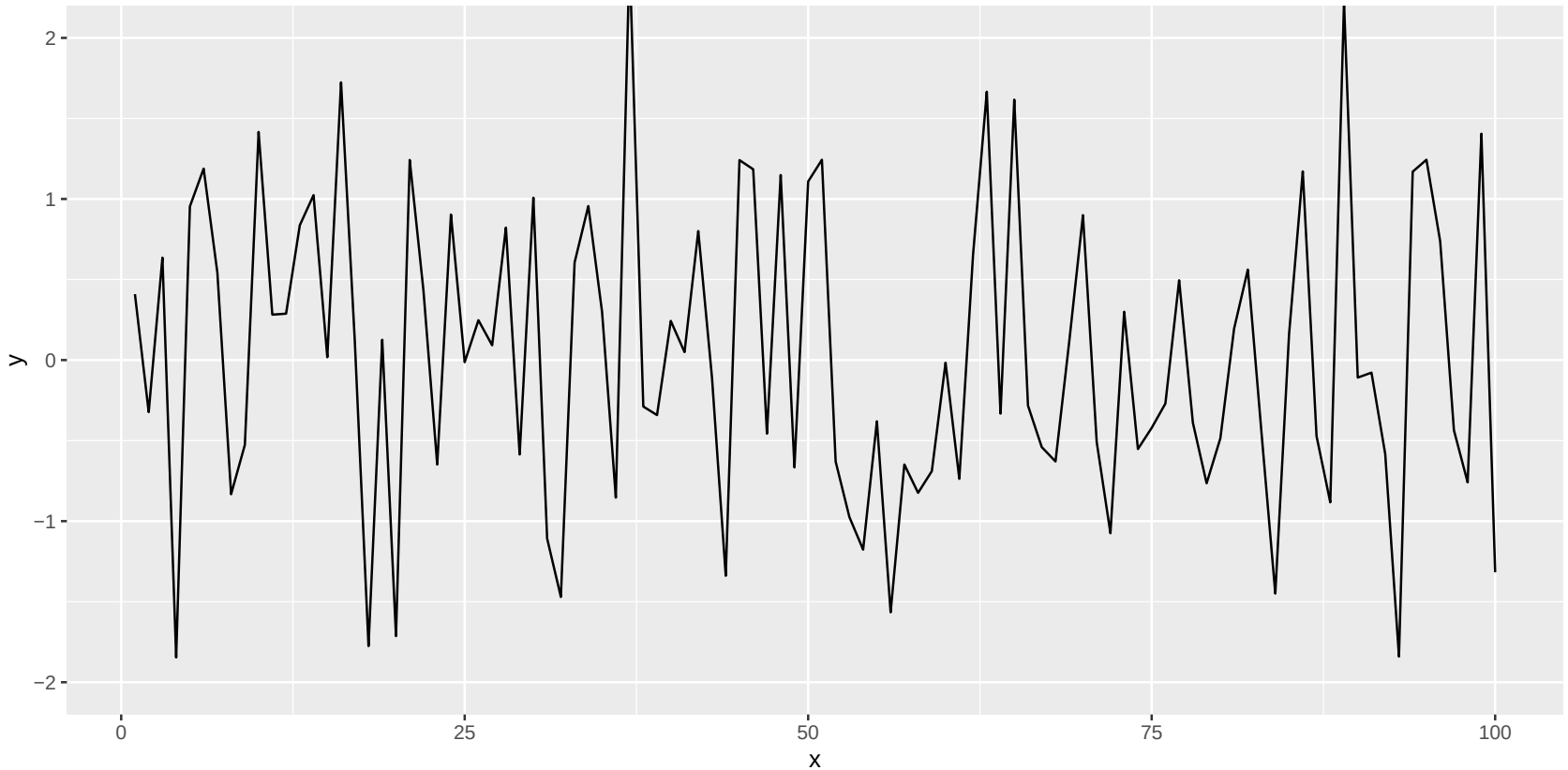
```
> set.seed(1984)
> x <- 1:100; y <- rnorm(100)
> ggplot(data.frame(x=x,y=y),aes(x,y)) + geom_line() + ylim(-2,2)
```




```
> plot(x,y,type="l",ylim=c(-2,2))
```



```
> ggplot(data.frame(x=x,y=y),aes(x,y)) + geom_line() +
+   coord_cartesian(ylim=c(-2,2))
```

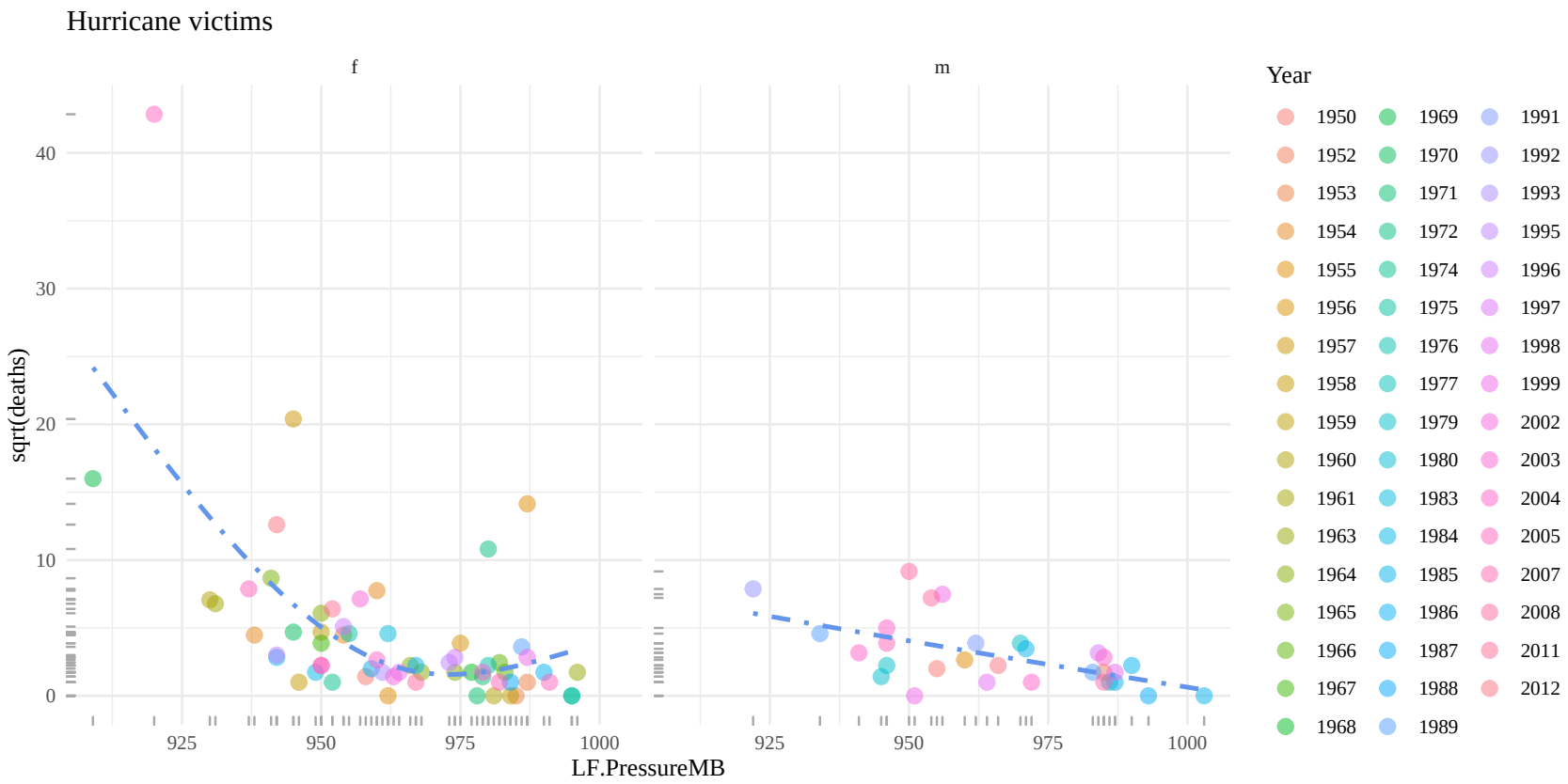


Final execise

hurricNamed data set from Vincent Arel-Bundock

```
##                    Name Year LF.WindsMPH LF.PressureMB LF.times
## Easy1950           Easy 1950         120           958        1
## King1950           King 1950         130           955        1
## Able1952           Able 1952          85           985        1
## Barbara1953     Barbara 1953          85           987        1
## Florence1953 Florence 1953          85           985        1
## Carol1954       Carol 1954         120           960        2
## Edna1954         Edna 1954         120           954        1
## Hazel1954       Hazel 1954         145           938        1
```

How to make this plot?



Final code

```
> d <- read.csv("hurricNamed.csv",row.names=1)
> d$Year <- factor(d$Year)
> ggplot(d,aes(LF.PressureMB,sqrt(deaths))) +
+   geom_point(aes(color=Year),size=3,alpha=0.5) +
+   geom_smooth(method="lm",se=F,alpha=0.7,size=1,linetype=4,
+               color="cornflowerblue",formula=y ~ splines::ns(x,2)) +
+   geom_rug(length=unit(0.05,"inch"),color="darkgray") +
+   facet_grid(. ~ mf) + theme_minimal(base_family="serif") +
+   labs(title="Hurricane victims")
```

Summary

- **Base** is very easy for prototype.
- The single function of **lattice** is very suitable for automatic reports
- **ggplot2** follows the *Grammar of Graphics*. It makes you understand the aesthetic mappings and plot a complex figure.

You can find more ggplot2 example from
https : //www.r − graph − gallery.com/index.html

Thank you